First, we have to edit spec/rails_helper.rb to require our new Capybara driver so that it's loaded:

```
require 'rspec/rails'
# Add additional requires below this line. Rails is not loaded until this…
require_relative './support/capybara.rb'
```

Often apps will auto-require all .rb files inside the spec/support directory. If that's the case for your app, this require could be omitted.

OK, what next? Recall that earlier, we used RSpec's configuration hooks to set rack_test as the default driver for our system tests. We now need to make it so that non-JavaScript system tests continue to use that, whereas JavaScript tests (indicated with the js: true tag in the test definition) should use our new Selenium Chrome Capybara driver.

We can achieve this by adding the following lines to spec/rails_helper.rb (add them in between the previous before hook we added and the final end):

```
Line 1 config.before(:each, type: :system, js: true) do
  2   driven_by :selenium_chrome_in_container
  3   Capybara.server_host = "0.0.0.0"
  4   Capybara.server_port = 4000
  5   Capybara.app_host = 'http://web:4000'
  6 end
```

System specs that specify js: true will use this new config (ones without will continue to use our old config, which sets the rack_test driver). This new config sets the Capybara driver (line 2) to selenium_chrome_in_container: our new driver that runs the tests via our remote Selenium Chrome browser.

Since the browser executing the tests will be running in a separate container, rather than on the same machine, some additional config is required. Capybara will start a new Puma server to run a test version of our app. Normally, this just happens on localhost on the same machine that the tests are running on, and everything just works. Here, however, this test version of our app needs to be accessible *externally*, from the Selenium Chrome container. That means we have to start the test app on a known port—I've chosen port 4000 (line 4), but you could choose anything. Also, much like when we started the Rails server with the -b 0.0.0.0 to listen on all ports, Capybara must start the test app server listening on all IP addresses. That's why we set server_host to 0.0.0.0 (line 3); without this, the server would be started on localhost, and only incoming requests within the container would be serviced.

Finally, we need to tell Capybara to use a URL for the app that will work when connecting from within the Selenium Chrome container (line 5). Remember,